Distributed LLM Inference on the Edge

Parallelism, Network and Bottlenecks on a Raspberry Pi Cluster

Group 7: Yuzhou Zhang, Yunfan Zhang, Shuo Zhang, Wenqi Zhao

DAT300/DIT616, Chalmers University of Technology
Supervisors: Yixing Zhang and Romaric Duvignau

Final presentation, October 2026

Motivation and research questions

- Cloud LLMs: private data leaves the device, costs money, needs connectivity
- One edge board: private and offline, but 4 GB RAM and a weak CPU
- **Idea:** let a cluster of cheap boards run the model together
- **Goal:** measure and profile it, find what limits it, remove the limits

RQ1 How do throughput and TTFT scale with the number of nodes?

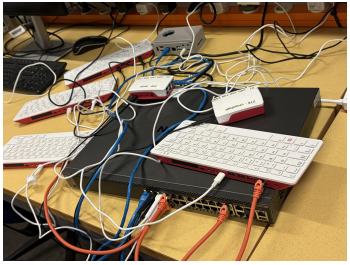
RQ2 Pipeline (PP) vs. tensor (TP) parallelism?

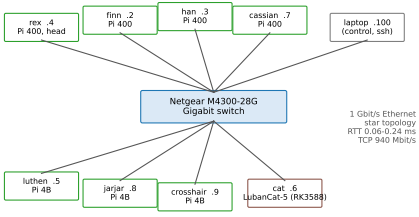
RQ3 What is the bottleneck: bandwidth, latency, memory, heterogeneity?

RQ4 Can simple optimizations help without hurting accuracy?

Workload: Llama-3.2-3B-Instruct, Q4_0 (1.9 GB)
Metrics: prefill and decode tokens/s, network traffic, CPU and memory per device

Cluster: 8 devices on one Gigabit switch



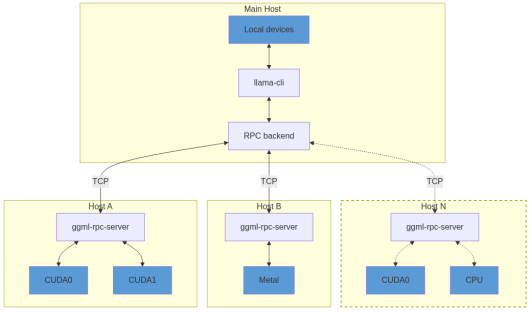


	Pi 400 / 4B (×7)	RK3588
CPU	4×A72	4×A76 + 4×A55
RAM	4 GB	4 GB
Mem. bandwidth	3.7–4.2 GB/s	21–23 GB/s
Decode alone	1.84 tok/s	12.28 tok/s

Network: 940 Mbit/s TCP, RTT 0.06–0.24 ms

A single board reads the weights at **70–85%** of its memory bandwidth ⇒ decode is memory-bound

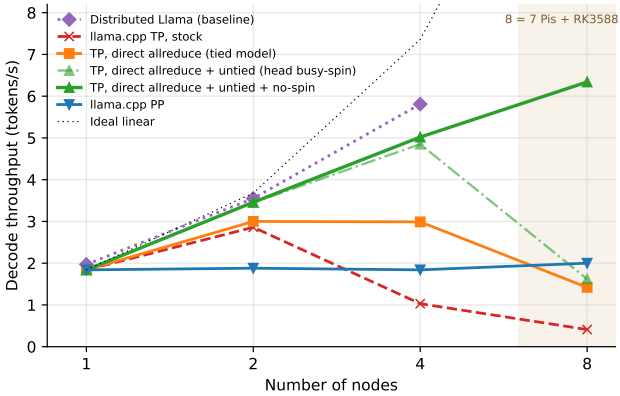
Frameworks and method



`llama.cpp` RPC backend (official diagram): `-sm layer` = PP, `-sm tensor` = TP, 56 allreduces per token

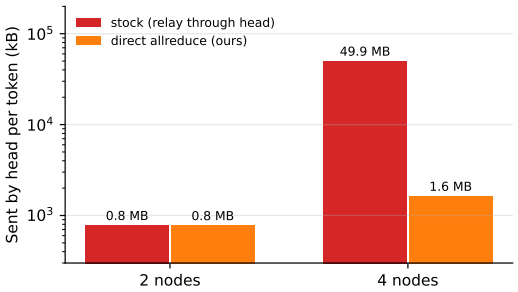
1. Baseline with **Distributed Llama** (TP only): 1.97 / 3.55 / 5.81 tok/s on 1 / 2 / 4 nodes
2. PP vs. TP with the **llama.cpp RPC backend**, 1–8 devices
3. Two instruments to attribute bottlenecks:
   - **resource monitor** on every device (CPU, memory, bytes, packets; 0.5 s)
   - **timing log** inside every RPC server (compute / sync / idle per token)
4. Fix the bottleneck, measure again, check perplexity

Result: decode throughput from 1 to 8 nodes



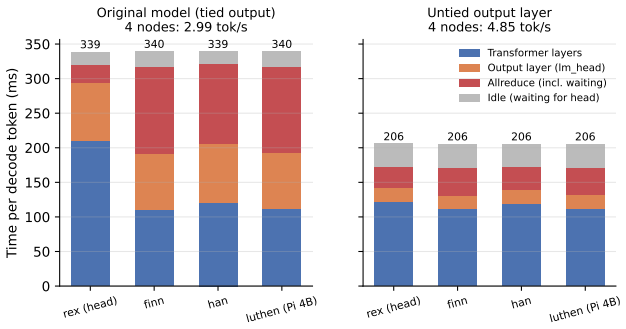
- **PP never beats one board** (1.84–2.00 tok/s): stages run one after another
- **Stock TP gets slower** with more nodes: 0.41 tok/s on 8
- Ours: **6.34 tok/s** on 8 nodes (3.4× one Pi, 15.5× stock)
- Prefill: 4.1 → 16.2 tok/s

Bottleneck 1: every allreduce relayed through the head



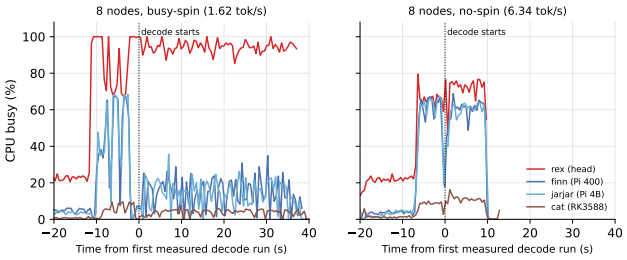
- Stock `llama.cpp`: direct exchange only for exactly 2 servers; otherwise the head downloads, sums and re-uploads every partial result
- 4 nodes: head sends **49.9 MB per token**, decode 1.03 tok/s (slower than one board)
- **Fix:** direct peer-to-peer **recursive-doubling allreduce**, $\log_2 N$ rounds of 12 KB
- 4 nodes: 1.03 $\rightarrow$ 2.99 tok/s

Bottleneck 2: the output layer is not split



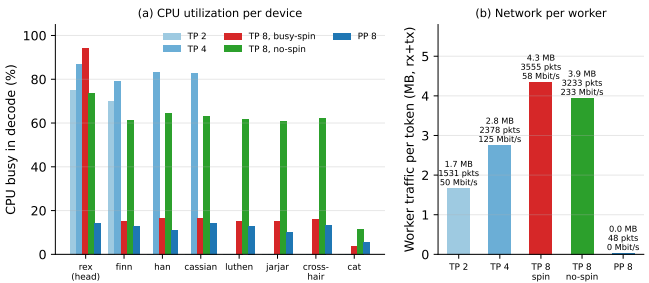
- Timing log: every node spends **80–85 ms/token** on the 323 MB output layer, regardless of N
- Cause: Llama-3.2 ties output and input embedding; `llama.cpp` replicates the tied tensor
- **Fix:** store an identical untied `output.weight` $\Rightarrow$ split N ways, **identical perplexity**
- 4 nodes: 2.99 $\rightarrow$ 4.85 tok/s (+62%)

Bottleneck 3: at 8 nodes the head's CPU is exhausted



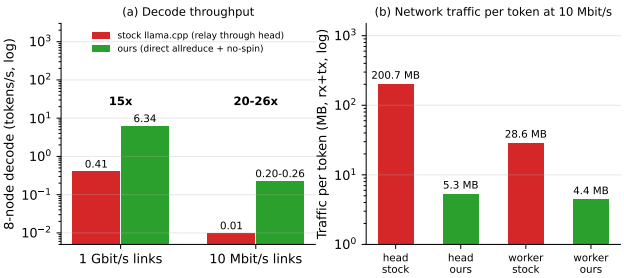
- Monitor: head CPU **94%**, workers 15%
- Timing log: every node **idle 470 of 631 ms** per token, waiting for the head
- gdb sampling: 8 **busy-spinning** dispatcher threads + 4 compute threads on 4 cores
- **Fix:** `GGML_RPC_NO_SPIN`: 1.62 → **6.34 tok/s**, idle 470 → 15 ms

The network is not the bottleneck (on a Gigabit switch)



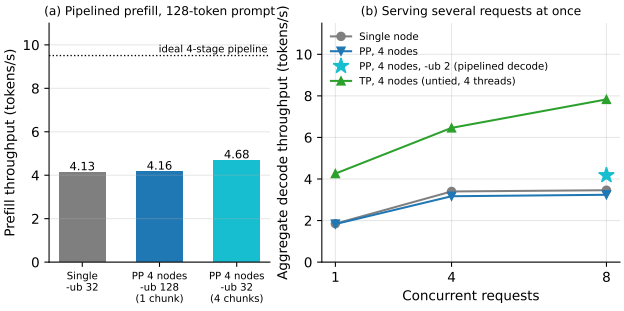
- 8 nodes: ≈4 MB and 3200–3500 packets per token per worker, at most **233 of 940 Mbit/s**
- Same traffic with and without busy-spin, but **3.9×** speed difference
- Bytes/token follow $2 \cdot 56 \cdot \log_2 N \cdot 12$ KB
- Memory is not limiting either (≈260 MB per worker at 8 nodes)

When the network is slow: links limited to 10 Mbit/s



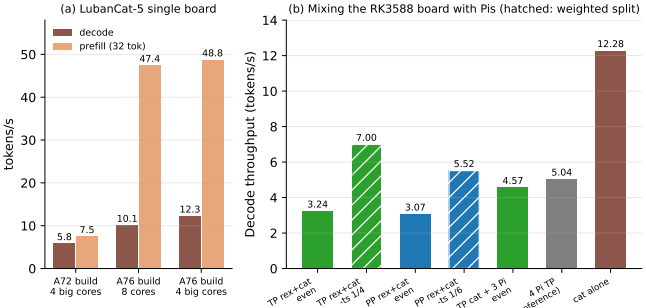
- 8 nodes, all device links at 10 Mbit/s (after loading the model)
- Stock: head moves **200 MB per token**, link saturated, **0.01 tok/s**
- Ours: 5.3 MB at the head, **0.20–0.26 tok/s**
- **20–26**× faster (15× at 1 Gbit/s): the traffic reduction matters most on slow networks

Pipeline parallelism: what can and cannot be improved



- One request: **never faster than one board** (by design)
- Chunked prefill (`-ub 32`): +12% (ideal 2.3×)
- Several requests + small chunks: 3.24 → **4.18 tok/s** (+29%), the only PP case above one board
- TP still wins: **7.83 tok/s** with 8 requests
- Real use of PP: models that do not fit one board

Heterogeneous devices: one RK3588 board



- Build for Cortex-A76 (dot-product instructions): 5.84 → **12.28 tok/s**
- Even split: fast board waits for the Pis
- Speed-proportional split (`-ts`): TP 2 nodes 3.24 → **7.00 tok/s**
- But the RK3588 alone is faster than any Pi mix for this model

Summary

Bottleneck	Optimization	Effect (decode, tok/s)
Single board: memory bandwidth	distribute the model	–
Allreduce relayed through head	direct recursive-doubling allreduce	4 nodes: 1.03 → 2.99
Tied output layer replicated	untied copy (identical output)	4 nodes: 2.99 → 4.85
Head CPU (busy-spin) at 8 nodes	`GGML_RPC_NO_SPIN`	8 nodes: 1.62 → 6.34
Slow network (10 Mbit/s)	direct allreduce	8 nodes: 0.01 → 0.20–0.26
PP: one stage active	batching + pipelined decode	3.24 → 4.18
Heterogeneity	A76 build, weighted split	3.24 → 7.00

- **RQ1/2:** TP scales to 3.4× on 8 nodes; PP only adds capacity
- **RQ3:** memory bandwidth on one board; on the cluster, whatever does not shrink with N (head relay, unsplit layer, head CPU), **not network bandwidth**
- **RQ4:** simple changes, no accuracy loss

Limitations and future work

Limitations

- 8 nodes always include the RK3588 board
- One model (3B); larger-than-board models not measured
- Weighted TP splits beyond 2 nodes fall back to the head relay
- 1–2 repetitions per configuration

Future work

- Uneven shards in the direct allreduce (weighted heterogeneous TP)
- Reduce the per-token head overhead and the number of sync rounds
- Direct activation transfer between PP stages
- Larger models; speculative decoding, KV-cache compression

Code, scripts and raw data: `https://github.com/44sunsetsf/dat300-edge-llm`

Thank you! Questions?